Sistemas Operativos (Caso de Aplicación)

Clase 2

Universidad Tecnológica Nacional - FRH

Técnicas Digitales III

Ciclo Lectivo 2020

Autor: Alejandro Furfaro / Mariano Mercado

IPC, Message Queue

- Se asemeja a una FIFO, salvo que pueden ser extraidos los datos en cualquier orden.
- Los mensajes estan compuestos por un header de tamaño fijo seguido por los datos a enviar de tamaño variable.
- El header de tamaño fijo (long) es el message type, y dicho valor nos permitira extraer selectivamente el dato de la queue.
- Una vez que un proceso lee un dato de la cola, el mismo es destruido, al igual que en los Pipes y FIFOs.
- Para crear o abrir una Message Queue se utiliza la system call *msgget (key, permisos)*

IPC, Message Queue

- Para enviar un mensaje se utiliza *msgsnd()* con los siguientes argumentos:
  - El ID de la message queue.
  - El tamaño del mensaje a ser enviado.
  - La dirección donde comienza el buffer que contiene el message type seguido del string a ser enviado.

- Para extraer un mensaje se utiliza msgrcv() con los siguientes argumentos:
  - El ID de la messague queue.
  - El puntero al buffer donde se almacenara el message type seguido del string leido.
  - El tamaño del mensaje a ser leido.
  - Un valor t que especifica el mensaje a ser leido

Si t vale 0, se extrae el primer mensaje de la cola (igual que las FIFO).

Si t es mayor a 0, se extrae el primer mensaje cuyo message type es igual a t.

Si t es menor a 0, se extrae el primer mensaje cuyo message type sea el minimo valor menor o igual al valor absoluto de t.

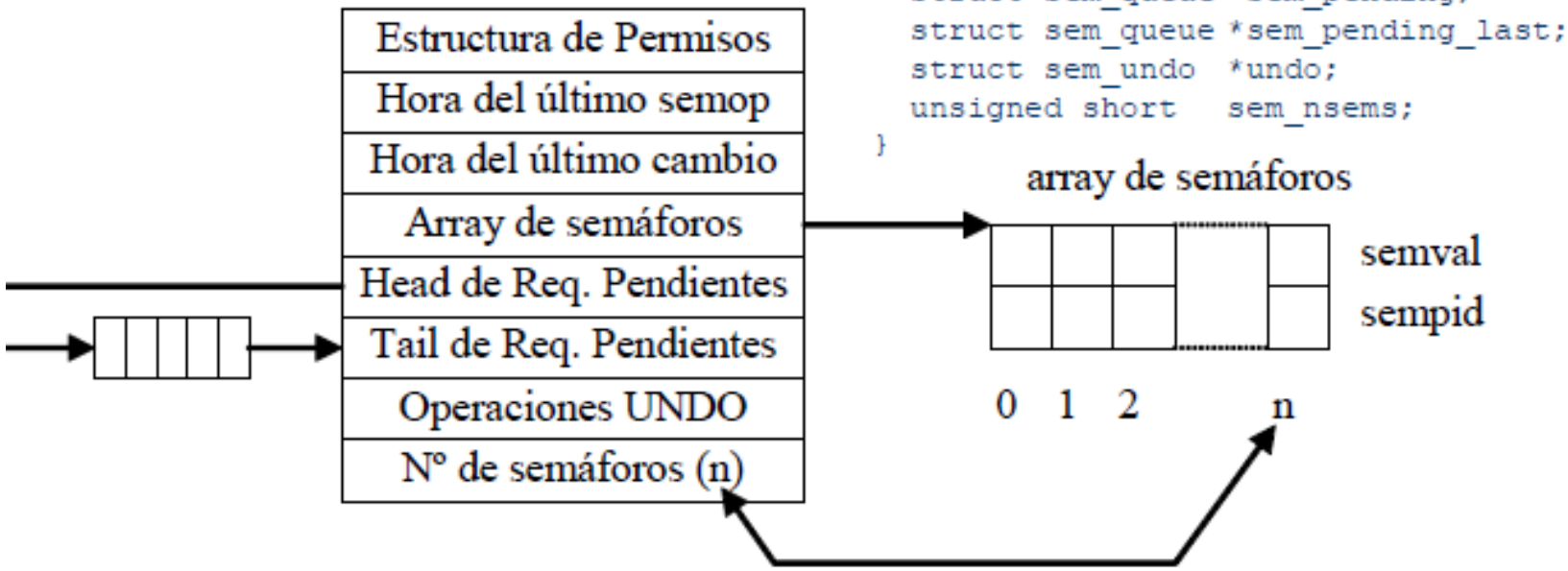
IPC, Semáforos

- Son contadores utilizados para controlar el acceso a estructuras de datos compartidas por multiples procesos.
- El Valor del semaforo es positivo si el recurso se encuentra disponible.
- Cada proceso que quiere acceder a dicho recurso decrementa en 1 el semaforo. Y cada vez que lo libera incrementa en 1 el mismo.
- El proceso solo puede utilizar el recurso si el semaforo es positivo (mayor a 0), si el mismo es negativo, espera hasta que el recurso se libera.
- Se utilizan las funciones *semget()* para crear un conjunto (set) de semaforos, *semop()* para verificar el valor del semaforo, para incrementar y para decrementar su valor.

IPC, Semáforos

La creación de un semáforo (de un set) genera una estructura de datos que permite administrar al mismo. Dicha estructura es la semid_ds:

```
struct semid_ds {
  struct ipc_perm  sem_perm;
  kernel_time_t    sem_otime;
  kernel_time_t    sem_ctime;
  struct sem       *sem_base;
  struct sem_queue *sem_pending;
  struct sem_queue *sem_pending_last;
  struct sem_undo  *undo;
  unsigned short   sem_nsems;
}
```



IPC, Semáforos
Creación - semget

semget permite crear un set de semáforos o tomar el control del mismo.

int semget (key_t key, int nsems, int semflg);

key_t key: *Utilizado para generar un identificador de semáforo único. El* valor key es obtenido previamente por la funcion ftok (si el set de semáforos es público) o puede contener el valor IPC_PRIVATE (si el set de semáforos solo es accesible por el proceso en curso).

int nsems: *Cantidad de semáforos en el set a crear.*

int semflg: *Especifica los permisos de acceso al semáforo por parte del* owner, del grupo y del resto "OR"eado con IPC_CREAT y/o IPC_EXCL.

Si se utiliza IPC_CREAT y IPC_EXCL al mismo tiempo y el semáforo no existe, éste se crea (pero no con acceso exclusivo !!). Si existe, retorna un error.

IPC, Semáforos
Creación - semget

Si la llamada a semget es exitosa se devuelve un valor positivo (identificador del semáforo), en cambio si es fallida se devuelve -1 y se guarda en *errno la causa* del error. La siguiente tabla muestra los errores que semget puede retornar:

Valor	Descripción
EOENT	Semaphore identifier does not exist for this key.
ENOMEM	Insufficient system memory to allocate the semaphore set.
EACCES	Semaphore identifier exists for this key, but requested operation is not allowed by current access permissions.
EEXIST	Semaphore identifier exists for this key, but the flags IPC_CREAT and IPC_EXCL are both set.
ENOSPC	System-imposed limit (SEMMNI) for the number of semaphore sets or systemwide maximum number of semaphores (SEMMNS) has been reached.
EIDRM	Specified semaphore set is marked for removal.

IPC, Semáforos
Configuración - semctl

La función semctl permite configurar los semáforos del set creado con semget, modificando el contenido de la estructura semid_ds

int semctl (int semid, int semnum, int cmd, union semun arg);

semid: Identificación del semáforo (obtenida de la función semget).
semnum: Número de semáforo (utilizado para los comandos que actúan sobre un semáforo en particular).
cmd: Comando a ser ejecutado.
arg: Unión cuyo contenido dependerá del comando ejecutado.

Valor retornado:
Error: Devuelve -1 y se setea ***errno*** con el error correspondiente.
Éxito: 0 o el valor solicitado.

```
union semun
{ int val;                    /* value for SETVAL */
struct semid_ds *buf;         /* buffer for IPC_STAT, IPC_SET */
unsigned short int *array;    /* array for GETALL, SETALL */
struct seminfo *__buf;        /* buffer for IPC_INFO */
};
```

IPC, Semáforos
Configuración - semctl

Comandos más utilizados:

IPC_STAT	Devuelve la estructura semid_ds en el buffer entregado en el 4to argumento. *(Variable en la union: struct semid_ds *buf).*
IPC_SET	Modifica los permisos definidos en la estructura semid_ds.sem_perm.
IPC_RMID	Remueve el semáforo.
GETALL	Retorna un array con los valores de cada semaforo. El array se guarda a partir del puntero entregado en el 4to argumento (Variable en la unión: unsigned short int *array).
SETALL	Setea los valores de los semáforos con los almacenados en el array entregado como 4to argumento.
GETVAL	Retorna el valor particular del semáforo indicado en semnum.
SETVAL	Setea el semáforo semnum con el valor indicado en el 4to 7 argumento (Variable en la unión: int val).

IPC, Semáforos
Operación - semop

Función semop:

int semop(int semid, struct sembuf *sops, unsigned nsops);

semid: Identificación del semáforo.
sops: Puntero al array que contiene los comandos a ser ejecutados para cada semáforo.
nsops: Cantidad de elementos del array enviado en sops.

Estructura sembuf:

```
struct sembuf
        {unsigned short int sem_num;    // semaphore #: 0 = first
        short int sem_op;               // semaphore operation
        short int sem_flg;              // operation flags
        };
```

IPC, Semáforos
Operación - semop

Descripción de la estructura sembuf

- sem_num: Número de semáforo (0 es el primer semáforo).
- sem_op: Operación a realizar en dicho semáforo.
- sem_flag: Flag que puede contener: IPC_NOWAIT o SEM_UNDO.

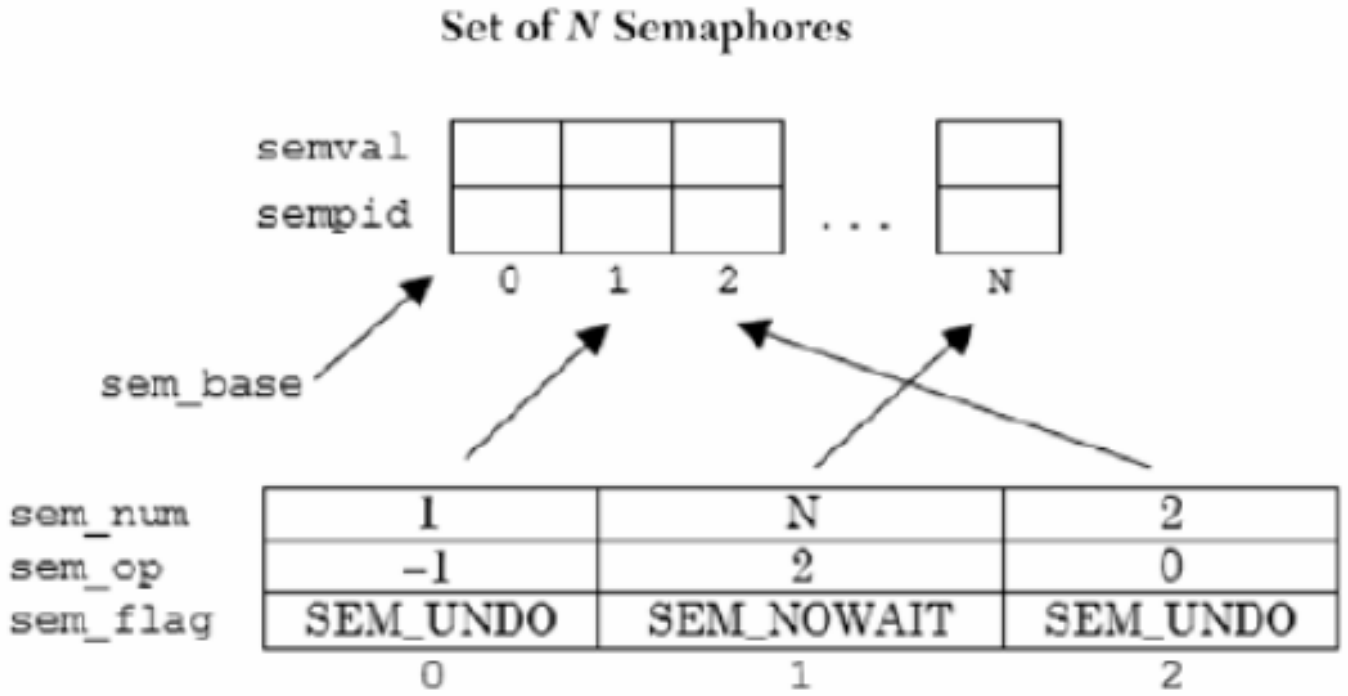
Descripción de los flags disponibles:

IPC_NOWAIT: Si la operación solicitada no puede ser ejecutada, se retorna el error correspondiente automáticamente (No bloqueante)

SEM_UNDO: Actualiza el valor del contador UNDO del semáforo, permitiendo deshacer esta operación. (IPC_NOWAIT no debe estar como flag).

IPC, Semáforos
Operación - semop

Valores de sem_op:
Si el valor de sem_op es positivo → semval = semval + sem_op.
Si el valor de sem_op es 0 y…
… semval = 0 → Se retorna inmediatamente de la función.
… semval != 0, IPC_NOWAIT → Se retorna -1 con errno=EAGAIN.
… semval !=0 → La función se bloquea hasta que semval valga 0



IPC, Semáforos
Operación - semop

Si el valor de sem_op es negativo y…
… semval >= abs(sem_op)
→ semval = semval – abs(semop)
… semval < abs(sem_op) y IPC_NOWAIT
→ Se retorna -1 con errno=EAGAIN.
… semval < abs(sem_op)
→ La función se bloquea hasta que semval>=abs(sem_op), y cuando esto sucede se decrementa el valor abs(sem_op) a semval; o hasta que se remueva el semáforo o hasta que se capture una señal (IPC: Signal).

Cuando semop se bloquea con un…
… sem_op = 0 → se incrementa el contador semzcnt (Procesos en espera de un 0 en el valor del semáforo).
… sem_op < 0 → se incrementa el contador semncnt (Procesos en espera de un incremento en el valor del semáforo)

IPC, Shared Memory

- Permite a dos procesos acceder a un mismo espacio de datos en memoria.
- Para crear una shared memory se utiliza la system call *shmget().*
- Luego se utiliza *shmat()* para “atachear” la shared memory al proceso.
- Finalmente un proceso “desatacha” la shared memory utilizando *shmdt()*.

Ejemplos:

```
    key_t key;
    int shmid;
    key = ftok("/home/beej/somefile3", 'R');
    shmid = shmget(key, 1024, 0644 | IPC_CREAT);
/* 1024: Tamaño de la shared memory: 1Kbyte */
```

IPC, Shared Memory

```
void * shmat(int shmid, void *shmaddr, int shmflg);
/* shmid:  Shared Memory ID
   shmaddr: 0 para que el S.O. asigne la posicion de
            memoria donde comienza la shared memory.
            Cualquier otro valor positivo permite setear
                 al usuario la direccion de comienzo.
   shmflg: SHM_RDONLY si es de solo lectura, o en caso
                contrario se inicializa en 0.

Valor retornado:  puntero void  ( void * )
*/
Ejemplo:
    char *data;
    key = ftok("/home/beej/somefile3", 'R');
    shmid = shmget(key, 1024, 0644 | IPC_CREAT);
    data = shmat(shmid, (void *)0, 0);
    gets(data);
    printf("shared contents: %s\n", data);
```